

SERVANTS OF BHARAT

Product Requirements Document

Volunteer Coordination · Milestone Tracking · Verified Certification

Table of Contents

Table of Contents	2
1. Overview	4
2. Problem Statement.....	4
3. Goals (V1).....	4
4. Users & Roles.....	4
5. Scope.....	5
5.1 In Scope (V1).....	5
5.2 Out of Scope (V1 — Reserved for Later Phases).....	5
6. Key Business Rules.....	5
7. User Journeys.....	6
7.1 Volunteer Flow.....	6
7.2 Admin Flow (Event Manager / Super Admin).....	6
7.3 External Reviewer Flow.....	7
8. Functional Requirements	7
8.1 Sign-Up & Auth.....	7
8.2 5-Star Milestone Tracker	7
8.3 Notice Board	7
8.4 Proof Submission & Review.....	7
8.5 Certification	8
8.6 Notifications	8
9. Data Model	8
organizations — reserved for Phase 2 (not used in V1 sign-up flow).....	8
users.....	9
service_categories — seeded once, not user-editable.....	9
events.....	10
submissions.....	10
submission_reviews	11
volunteer_progress	11
certificates.....	12
notifications	12
Phase 2 tables (present in schema now, not wired into V1 logic).....	12
volunteer_events	12
partner_organizations & event_partners.....	13
donors & donations.....	13
10. API Design (Suggested REST Endpoints)	13
Auth	13
Profile.....	14
Milestone Tracker.....	14
Notice Board.....	14
Submissions	14

Certificates.....	14
Notifications	14
Admin — Users.....	14
11. Screens / Pages	14
Public.....	14
Volunteer.....	15
Admin (Event Manager / Super Admin).....	15
12. Non-Functional Requirements.....	15
13. Tech Stack (Confirmed).....	15
14. Third-Party Integrations.....	16
15. Roadmap.....	16
16. Success Metrics	17
17. Risks & Mitigations	17
18. Phase 2 — Detailed Specification.....	17
18.1 Event Registration & Attendance (volunteer_events).....	17
Functional Requirements.....	17
New API Endpoints.....	18
New/Updated Screens.....	18
18.2 Partner Organisations (partner_organizations, event_partners)	18
Functional Requirements.....	18
New API Endpoints.....	18
New/Updated Screens.....	18
18.3 Donor & Donation Management (donors, donations).....	19
Functional Requirements.....	19
New API Endpoints.....	19
New/Updated Screens.....	19
18.4 Gen AI Review Assistance (LangChain).....	19
Functional Requirements.....	19
New API Endpoints.....	19
New/Updated Screens.....	20
19. QA & Testing Strategy.....	20
19.1 Automated Functional Testing — Python + Robot Framework.....	20
19.2 Performance/Load Testing — JMeter	20

1. Overview

Servants of Bharat is a web platform that replaces informal WhatsApp-based volunteer coordination with one structured system. Volunteers discover admin-posted service events, submit photo proof of their work, and track progress toward completing 5 milestone stars in a service year. Admin-type users review and approve or reject that proof from a single dashboard.

On completing all 5 stars, the volunteer automatically receives a QR-verifiable digital certificate by email.

Anyone can sign up — no college, company, or organisational affiliation is required.

2. Problem Statement

- Coordination is fragmented across unlinked WhatsApp groups; no single source of truth.
- Proof photos pile up unsorted on coordinators' personal phones.
- Volunteers miss local events because they weren't added to the right chat group.
- There is no verifiable, tamper-proof record of who completed what.
- Manual certificate issuance is slow and inconsistent.

3. Goals (V1)

1. One structured, always-current website instead of WhatsApp groups.
2. A clear, visual goal for every volunteer: 5 milestone stars in a year.
3. One review workspace for admins instead of scattered phone galleries.
4. A verifiable, auditable record of completed service.
5. Automatic certificate generation and delivery the moment the 5th star is approved.
6. Open sign-up — no organisation required.

4. Users & Roles

Role	Enum Value	Who	What They Do
Volunteer	VOLUNTEER	Any registered individual	Browses events, submits proof, tracks own progress, receives certificate
Event Manager	EVENT_MANAGER	Admin team member	Posts events, reviews submissions, approves/rejects proof

Super Admin	SUPER_ADMIN	Admin team member with full access	Everything Event Manager can do, plus manages users (block/unblock), views all data
External Reviewer	(not a login role)	Anyone with a certificate's QR code / verification link	Views a public, read-only certificate verification page

Out of scope for V1 (no login role, no screens): Donor, Partner Organisation.

5. Scope

5.1 In Scope (V1)

- Volunteer sign-up, login, profile.
- 5-star annual milestone tracker: Women's Care, Child Welfare, Elder Care & Orphanage, Environmental Plantation, Blood Donation.
- Admin-managed public notice board with category filters and auto-archiving of past events.
- Photo + note proof submission, optionally linked to a notice-board event.
- Single centralized admin review queue with Approve / Reject (structured reasons).
- Automatic certificate generation, light customisation step, email delivery, QR verification.
- In-app notifications for approval/rejection/certificate events.
- Seasonal (quarterly) submission limit to space out field-based milestones.

5.2 Out of Scope (V1 — Reserved for Later Phases)

- Event RSVP, check-in codes, or attendance tracking.
- Automated location verification or AI-based photo authenticity checks (manual-only in V1).
- Donor portal / impact dashboards.
- Partner organisation co-hosting features.
- Multi-region/language rollout.

6. Key Business Rules

These rules must be enforced server-side, not just in the UI:

7. Fixed categories. Exactly 5 categories exist, seeded once: Women's Care, Child Welfare, Elder Care & Orphanage, Environmental Plantation, Blood Donation.
8. One star per category per service year. A volunteer can hold at most one APPROVED progress record per (volunteer_id, category_id, service_year).
9. Locking. Once a submission is APPROVED and its volunteer_progress row is marked completed = true, that star cannot be re-submitted or reversed through normal app flows.

10. Quarterly limit (seasonal restriction). A volunteer may have at most one submission with quarter = Q per service_year for field-based categories (Women's Care, Child Welfare, Elder Care & Orphanage, Environmental Plantation). Blood Donation is exempt from the quarterly limit since it can be completed at any point in the year. A REJECTED submission does not count against the quarter limit — the volunteer may resubmit within the same quarter.
11. Service year definition (confirmed). The service year is the calendar year, January 1 – December 31. Quarters map directly to calendar quarters: Q1 = Jan–Mar, Q2 = Apr–Jun, Q3 = Jul–Sep, Q4 = Oct–Dec. service_year is simply the calendar year in which a submission is made, and all 5-star progress resets/tracks per calendar year.
12. Organisation is optional and free text (confirmed). Sign-up never blocks on organisation. If the volunteer has one, they type its name directly into a text field (users.organization_name) — there is no dropdown, no matching against the organizations table, and no validation. The organizations table is not used in the V1 signup flow (see Section 9).
13. Certificate trigger. The instant a volunteer's 5th distinct category is marked completed = true, the system flags them as programme-complete and prompts the one-screen certificate customisation step described in Section 8.5.
14. Immutable certificate record. Once issued, a certificates row is created with a unique certificate_number and verification_code; it is never re-issued for the same volunteer/year.
15. Role permissions. VOLUNTEER can only read/write their own data. EVENT_MANAGER and SUPER_ADMIN can manage events and review submissions. Only SUPER_ADMIN can change another user's status (ACTIVE/INACTIVE/BLOCKED).
16. Rejection notifications carry the exact reason (confirmed). When an admin rejects a submission, the volunteer's notification and email must show the exact reason text selected by the admin (e.g., "Blurry Image") plus any free-text remarks — never a generic "your submission was rejected" message with no reason.

7. User Journeys

7.1 Volunteer Flow

Sign up (name, phone, email, password, sex, date of birth, location, blood group, organisation optional) → Log in → See 5 empty star outlines → Browse/filter notice board → Attend an event in real life → Upload proof photo + note (optionally tag the event) → Wait for admin review → Get notified of approval/rejection → Approved star locks solid → Repeat until 5/5 → Certificate customisation screen → Certificate emailed automatically with QR code.

7.2 Admin Flow (Event Manager / Super Admin)

Log in → Post event (title, description, category, date/time, venue/address) → Monitor pending-submissions queue → Open a submission → View photo + note → Approve (star instantly credited) or Reject (pick a reason from a preset list) → Volunteer is notified automatically → Past events auto-archive.

7.3 External Reviewer Flow

Scan QR code on a printed/digital certificate → Land on a public verification page → See volunteer name, certificate number, issue date, and a "Valid" confirmation (no login required).

8. Functional Requirements

8.1 Sign-Up & Auth

- Fields: full name, phone (mandatory, unique-ish but not strictly enforced), email (unique, mandatory), password (hashed), sex, date of birth, location (free text or city/state), blood group, organisation name (optional, plain free-text field — no dropdown, no validation).
- Secure login (email + password) with hashed storage (bcrypt/argon2) and session/JWT-based auth.
- Password reset via email link.
- Role assigned at creation: default VOLUNTEER. Admin accounts (EVENT_MANAGER, SUPER_ADMIN) are created by a Super Admin, not via public sign-up.

8.2 5-Star Milestone Tracker

- Home screen shows 5 star icons — outline (not started), pending (submitted, awaiting review), solid/locked (approved).
- Tapping an empty star opens the upload flow for that category.
- Countdown widget showing days remaining in the current quarter (for field-based categories).
- Progress is scoped to a service_year (e.g., calendar year or programme year — configurable constant).

8.3 Notice Board

- Chronological list of upcoming events, newest/soonest first.
- Filter by category.
- Each event card: title, category, date/time, venue/address, short description.
- Admin event form: title, description, category, date, start/end time, venue, address, city, state, capacity (optional).
- Events auto-move to an "archived"/COMPLETED state once event_date has passed (a daily scheduled job, or computed at read-time).

8.4 Proof Submission & Review

- Volunteer submission form: choose category (only categories not yet completed this year, and not blocked by the quarter rule, are selectable), photo upload, text note, optional link to a notice-board event.
- Submission enters PENDING state in a single admin queue, visible to all EVENT_MANAGER/SUPER_ADMIN users.
- Admin queue: filter by category/status, sort by submitted date.
- Approve: one click → submissions.status = APPROVED, creates/updates volunteer_progress row with completed = true, completed_at = now().

- Reject: pick from a pre-set reason list (e.g., "Blurry Image", "Wrong Season/Quarter", "Not Related to Category", "Duplicate Submission", "Other") + optional free-text remark. The volunteer's notification and email must display this exact reason text (plus remarks, if any) — not a generic rejection message. Volunteer can resubmit for the same category (subject to Rule 4).
- Every review writes a `submission_reviews` row (who reviewed, when, reason if rejected).

8.5 Certification

- On 5th completed category: show the volunteer a one-time customisation screen — confirm display name, toggle "include organisation name" (only shown if `users.organization_name` is filled in), preview.
- On confirmation, generate a PDF certificate (name, organisation if included, completion date, unique certificate number) and a QR code linking to the public verification page.
- Email the certificate PDF to the volunteer automatically.
- Public verification page at `/verify/:verification_code` shows validity + basic details, no login required.

8.6 Notifications

- In-app notification created whenever: a submission is approved, a submission is rejected, a certificate is issued, or an admin posts a new event matching filters the volunteer has viewed (optional — keep simple: at minimum, approval/rejection/certificate).
- Rejection notifications must include the exact reason the admin selected (e.g., "Blurry Image") plus any remarks, embedded directly in `notifications.message` — never a generic "rejected" message.
- Notification bell with unread count; mark-as-read on open.

9. Data Model

organizations — reserved for Phase 2 (not used in V1 sign-up flow)

Column	Type	Notes
<code>id</code>	UUID	PK
<code>organization_id</code>	VARCHAR(50)	UNIQUE
<code>organization_name</code>	VARCHAR(200)	NOT NULL
<code>organization_type</code>	ENUM	COLLEGE, UNIVERSITY, SCHOOL, NGO, OTHER
<code>city</code>	VARCHAR(100)	
<code>state</code>	VARCHAR(100)	
<code>created_at</code>	TIMESTAMPTZ	DEFAULT now()

users

Column	Type	Notes
id	UUID	PK
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL
phone	VARCHAR(15)	
sex	VARCHAR(20)	
date_of_birth	DATE	
location	VARCHAR(150)	
blood_group	VARCHAR(5)	
organization_name	VARCHAR(200)	NULL — free text, typed directly by volunteer at sign-up; not a FK
role	ENUM	SUPER_ADMIN, EVENT_MANAGER, VOLUNTEER — NOT NULL, DEFAULT 'VOLUNTEER'
status	ENUM	ACTIVE, INACTIVE, BLOCKED — DEFAULT 'ACTIVE'
created_at	TIMESTAMPTZ	DEFAULT now()
updated_at	TIMESTAMPTZ	DEFAULT now()

service_categories — seeded once, not user-editable

Column	Type	Notes
id	SMALLSERIAL	PK
name	VARCHAR(100)	UNIQUE
description	TEXT	
display_order	SMALLINT	

Seed rows: 1 Women's Care · 2 Child Welfare · 3 Elder Care & Orphanage · 4 Environmental Plantation · 5 Blood Donation

events

Column	Type	Notes
id	UUID	PK
title	VARCHAR(200)	NOT NULL
description	TEXT	
category_id	SMALLINT	FK → service_categories(id)
venue	VARCHAR(200)	
address	TEXT	
city	VARCHAR(100)	
state	VARCHAR(100)	
event_date	DATE	NOT NULL
start_time / end_time	TIME	
capacity	INTEGER	
status	ENUM	UPCOMING, ONGOING, COMPLETED, CANCELLED
created_by	UUID	FK → users(id)
created_at / updated_at	TIMESTAMPTZ	DEFAULT now()

submissions

Column	Type	Notes
id	UUID	PK
volunteer_id	UUID	FK → users(id); must have role VOLUNTEER
event_id	UUID	FK → events(id), NULL — optional link
category_id	SMALLINT	FK → service_categories(id)
image_url	TEXT	NOT NULL
note	TEXT	
service_year	SMALLINT	NOT NULL

quarter	SMALLINT	CHECK (quarter BETWEEN 1 AND 4)
status	ENUM	PENDING, APPROVED, REJECTED — DEFAULT 'PENDING'
submitted_at / updated_at	TIMESTAMPTZ	DEFAULT now()

submission_reviews

Column	Type	Notes
id	UUID	PK
submission_id	UUID	FK → submissions(id), UNIQUE
reviewed_by	UUID	FK → users(id); must have role SUPER_ADMIN or EVENT_MANAGER
rejection_reason	VARCHAR(200)	NULL
remarks	TEXT	NULL
reviewed_at	TIMESTAMPTZ	DEFAULT now()

volunteer_progress

Column	Type	Notes
id	UUID	PK
volunteer_id	UUID	FK → users(id)
category_id	SMALLINT	FK → service_categories(id)
submission_id	UUID	FK → submissions(id)
service_year	SMALLINT	
completed	BOOLEAN	DEFAULT false
completed_at	DATE	
— constraint		UNIQUE (volunteer_id, category_id, service_year)

certificates

Column	Type	Notes
id	UUID	PK
volunteer_id	UUID	FK → users(id), UNIQUE
certificate_number	VARCHAR(50)	UNIQUE
verification_code	VARCHAR(100)	UNIQUE
pdf_url	TEXT	
qr_code_url	TEXT	
issued_at	TIMESTAMPTZ	

notifications

Column	Type	Notes
id	UUID	PK
user_id	UUID	FK → users(id)
title	VARCHAR(150)	
message	TEXT	
type	ENUM	EVENT, APPROVAL, REJECTION, CERTIFICATE, SYSTEM
is_read	BOOLEAN	DEFAULT false
created_at	TIMESTAMPTZ	DEFAULT now()

Phase 2 tables (present in schema now, not wired into V1 logic)

Full functional spec for these is in Section 18.

volunteer_events

Column	Type	Notes
id	UUID	PK
volunteer_id	UUID	FK → users(id); must have role VOLUNTEER
event_id	UUID	FK → events(id)

status	ENUM	REGISTERED, ATTENDED, ABSENT, CANCELLED — DEFAULT 'REGISTERED'
registered_at	TIMESTAMPTZ	DEFAULT now()
attended_at	TIMESTAMPTZ	NULL
remarks	TEXT	NULL
— constraint		UNIQUE (volunteer_id, event_id)

partner_organizations & event_partners

Column	Type	Notes
id	UUID	PK (partner_organizations)
name	VARCHAR(200)	NOT NULL
contact_person	VARCHAR(100)	
email / phone / website	VARCHAR	
address	TEXT	
created_at	TIMESTAMPTZ	DEFAULT now()
event_id + partner_id	UUID, UUID	event_partners — composite PK, FKs to events(id) and partner_organizations(id)

donors & donations

Column	Type	Notes
id	UUID	PK (donors)
name / organization / email / phone	VARCHAR	donors table
created_at	TIMESTAMPTZ	DEFAULT now()
donor_id	UUID	donations — FK → donors(id)
amount	NUMERIC(12,2)	NOT NULL
remarks	TEXT	
donated_at	TIMESTAMPTZ	DEFAULT now()

10. API Design (Suggested REST Endpoints)

Auth

POST /api/auth/signup

POST /api/auth/login
POST /api/auth/forgot-password
POST /api/auth/reset-password

Profile

GET /api/me PATCH
/api/me

Milestone Tracker

GET /api/me/progress – 5-star status for current service_year

Notice Board

GET /api/events?category=&status=
GET /api/events/:id
POST /api/events – EVENT_MANAGER, SUPER_ADMIN
PATCH /api/events/:id – EVENT_MANAGER, SUPER_ADMIN
DELETE /api/events/:id – SUPER_ADMIN

Submissions

POST /api/submissions – volunteer creates (multipart: image + fields)
GET /api/me/submissions
GET /api/admin/submissions?status=PENDING&category= – admin queue
POST /api/admin/submissions/:id/approve
POST /api/admin/submissions/:id/reject – body: { reason, remarks }

Certificates

GET /api/me/certificate
POST /api/me/certificate/customize – body: { includeOrganization }
GET /api/verify/:verification_code – public, no auth

Notifications

GET /api/me/notifications
PATCH /api/me/notifications/:id/read

Admin — Users

GET /api/admin/users – SUPER_ADMIN
PATCH /api/admin/users/:id/status – SUPER_ADMIN

11. Screens / Pages

Public

- Landing page
- Sign up / Log in / Forgot password
- Certificate verification page (/verify/:code)

Volunteer

- Home (5-star tracker + season countdown)
- Notice board (list + category filter)
- Event detail
- Submit proof (per category)
- My submissions (status history)
- Notifications
- Certificate page (view/download once issued)
- Profile / edit profile

Admin (Event Manager / Super Admin)

- Admin dashboard (pending count, quick stats)
- Post/edit event
- Review queue (list + detail with Approve/Reject)
- Manage users (Super Admin only)

12. Non-Functional Requirements

- Responsive design — usable on mobile and desktop browsers.
- Photos and personal data encrypted at rest; HTTPS everywhere.
- Simple, minimal-typing UI for digitally less-experienced users.
- Role-based access control enforced on every API route, not just hidden in UI.
- Reasonable performance target: review queue and notice board should load in under 2 seconds for a few thousand volunteers/events.

13. Tech Stack (Confirmed)

Layer	Choice	Notes for Implementation
Frontend	ReactJS	Vite + React, React Router for pages in Section 11, Axios/Fetch for the REST API in Section 10. Plain CSS/Tailwind for styling — keep components simple per the "minimal-typing UI" requirement.
Backend	Python (Flask)	Flask app factory pattern; Blueprints per module (auth, events, submissions, certificates, notifications, admin). Flask-JWT-Extended for auth, Flask-SQLAlchemy as ORM, Flask-Migrate (Alembic) for migrations, Marshmallow or Pydantic for request/response validation.

Database	PostgreSQL	Schema exactly as in Section 9 (plus Phase 2 tables, unused in V1). Use Alembic migrations generated from the SQLAlchemy models.
Gen AI	LangChain	Phase 2 feature — see Section 18.4. Not required to ship V1, but the backend should be structured (service-layer separation) so a LangChain-based review-assistant module can be added later without refactoring core submission logic.
Automation (QA)	Python + Robot Framework	End-to-end test suites for the volunteer flow (sign-up → submit → get approved → certificate) and the admin flow (post event → review → approve/reject). See Section 19.
NFR / Performance Testing	JMeter	Load/stress test plans for the notice board, submission upload, and admin review-queue endpoints. See Section 19.
File Storage	S3-compatible bucket (e.g., AWS S3, or MinIO for local dev)	Proof photos and generated certificate PDFs.
Email	SendGrid / Amazon SES (via Python SDK)	Certificate and notification emails.
PDF Generation	ReportLab or WeasyPrint (Python)	Renders the certificate template into a PDF.
QR Generation	qrcode Python package	Generates a PNG linked to the /verify/:verification_code URL.
Auth	JWT (Flask-JWT-Extended) + werkzeug.security / bcrypt	Password hashing.
Scheduled Job	APScheduler or cron calling a Flask CLI command	Marks past events as COMPLETED daily.

14. Third-Party Integrations

- Transactional email service (certificates, notifications).
- QR code generation.
- Cloud object storage for photos and certificate PDFs.

15. Roadmap

- Phase 1 — MVP (this document): Sign-up/login, 5-star tracker, notice board, proof submission, admin review, automatic certification.
- Phase 2 — Better Insights & Expanded Users: Richer admin analytics, LangChain-based AI review assistance, event RSVP/attendance tracking, Donor and Partner Organisation portals. Full spec in Section 18.
- Phase 3 — Scale: Multi-region rollout, broader integrations.

16. Success Metrics

- Number of volunteers completing all 5 milestones per programme year.
- Average admin review turnaround time.
- Reduction in manual WhatsApp coordination effort.
- Volunteer retention across consecutive seasons/years.
- Certificates issued and independently QR-verified.

17. Risks & Mitigations

Risk	Mitigation
Fake/reused proof photos	Manual admin review + structured rejection reasons; AI-assisted checks planned for Phase 2
Admin review backlog as volunteers grow	Centralized queue with clear filter/sort; add more EVENT_MANAGER accounts as needed
Low volunteer engagement	Visible progress board, season countdown, instant certificate reward
Data privacy	Encrypted storage, role-based access, minimal required data collection

18. Phase 2 — Detailed Specification

Phase 2 builds directly on top of the V1 schema and adds four capability areas. None of this is required to ship V1, but it's specified in full now so the team can build straight into it later without re-architecting.

18.1 Event Registration & Attendance (volunteer_events)

Purpose: Let volunteers formally RSVP to a notice-board event and let admins record who actually attended, instead of proof photos being the only signal.

Functional Requirements

- Volunteer can tap "I'm attending" on an event detail page → creates a volunteer_events row with status = REGISTERED.
- Volunteer can cancel their RSVP before the event date → status = CANCELLED.
- On/after the event date, an admin (or the volunteer via self check-in, if later enabled) marks each registrant ATTENDED or ABSENT.
- Event detail page (admin view) shows a registrant list with counts (Registered / Attended / Absent).

- A submission can optionally be cross-checked against attendance (e.g., flag submissions from volunteers who weren't marked ATTENDED) — surfaced as a soft warning to the reviewer, not a hard block, to keep manual review authority with the admin.

New API Endpoints

POST /api/events/:id/rsvp – volunteer registers
 DELETE /api/events/:id/rsvp – volunteer cancels
 GET /api/events/:id/registrants – admin: list of volunteer_events for an event
 PATCH /api/events/:id/registrants/:volunteer_id – admin sets ATTENDED/ABSENT

New/Updated Screens

- Event detail (volunteer): add "I'm attending" / "Cancel RSVP" button.
- Event detail (admin): registrant list with attendance marking.
- My events (volunteer): list of events they've RSVP'd to, past and upcoming.

18.2 Partner Organisations (partner_organizations, event_partners)

Purpose: Let colleges, CSR companies, or allied NGOs co-host events and get credit/visibility for it, and let them pull verified outcome data for their own reporting.

Functional Requirements

- Super Admin maintains a directory of partner organisations (name, contact person, email, phone, website, address).
- When creating/editing an event, an admin can tag one or more partner organisations as co-hosts (event_partners).
- Events co-hosted by a partner show a "Co-hosted by [Partner Name]" badge on the public notice board.
- A lightweight, read-only partner login (or a magic-link/report-export, whichever is simpler to build) lets a partner view: events they co-hosted, and aggregate completion counts tied to those events (no access to individual volunteer personal data — aggregate numbers only, to protect privacy).

New API Endpoints

GET /api/admin/partners
 POST /api/admin/partners – SUPER_ADMIN
 PATCH /api/admin/partners/:id – SUPER_ADMIN
 POST /api/admin/events/:id/partners – tag a partner as co-host – EVENT_MANAGER/SUPER_ADMIN
 DELETE /api/admin/events/:id/partners/:partner_id
 GET /api/partner/events – partner-facing, scoped to their own co-hosted events
 GET /api/partner/impact-summary – aggregate counts only, no PII

New/Updated Screens

- Admin: Partner directory (list/add/edit).
- Admin: event form gains a "Co-hosting partners" multi-select.
- Public notice board: co-host badge on event cards.
- Partner portal: their events + aggregate impact numbers.

18.3 Donor & Donation Management (donors, donations)

Purpose: Give the organisation a simple internal record of who has donated, and let the Admin show high-level, non-PII impact numbers to build donor trust — addressing the "no centralized, verifiable dashboard" pain point called out in research.

Functional Requirements

- Super Admin can record a donor (name, organisation if any, email, phone) and log donations against them (amount, date, remarks). This is an internal bookkeeping tool, not a payment gateway — no online payment processing in Phase 2.
- Admin dashboard gains an aggregate "Impact & Funding" panel: total certified volunteers, total milestones completed, total donations received (sum), shown as numbers/simple charts — never a public payment flow.
- Optionally, a read-only public "Impact" page can show the same aggregate numbers (no donor PII, no perdonation detail) for transparency, matching the goal of a "trusted, data-backed proof layer."

New API Endpoints

```
GET /api/admin/donors
POST /api/admin/donors    - SUPER_ADMIN
POST /api/admin/donors/:id/donations - SUPER_ADMIN, log a donation
GET /api/admin/impact-summary - aggregate counts for the admin dashboard
GET /api/impact    - public, aggregate-only, no auth
```

New/Updated Screens

- Admin: Donor list + add donor/donation form.
- Admin dashboard: Impact & Funding summary panel.
- Public (optional): Impact page.

18.4 Gen AI Review Assistance (LangChain)

Purpose: Speed up the admin review queue, directly addressing the "AI-assisted image review support" goal and the burnout/duplicate-image pain point, without removing the human-in-the-loop approval the organisation wants to keep.

Functional Requirements

- A LangChain-based service reads each pending submission's photo + note and produces a non-binding advisory summary for the reviewer: a one-line description of what the image appears to show, whether the note text plausibly matches the selected category, and a flag if the image looks like a likely duplicate of a prior submission from the same volunteer (via basic perceptual-hash or embedding similarity, orchestrated through a LangChain chain/tool).
- This advisory appears as a sidebar note next to the submission in the review queue — the Approve/Reject decision is always made by the human admin; the AI never auto-approves or auto-rejects.
- A simple admin-facing chat assistant (LangChain agent with read access to events/submissions data) can answer questions like "how many pending submissions are in Elder Care this month?" to speed up ad-hoc admin queries.

New API Endpoints

```
GET /api/admin/submissions/:id/ai-review - returns the advisory summary (generated on demand or cached)
```

POST /api/admin/assistant/query – admin chat assistant

New/Updated Screens

- Admin review queue: AI advisory panel next to each submission.
- Admin: simple assistant chat widget.

19. QA & Testing Strategy

19.1 Automated Functional Testing — Python + Robot Framework

- Robot Framework test suites (using RequestsLibrary for API-level tests and SeleniumLibrary/Browser library for UI-level tests) covering:
 - Volunteer flow: sign-up → login → browse notice board → submit proof → get notified on approval/rejection → reach 5/5 → receive certificate.
 - Admin flow: login → post event → review queue → approve/reject with reason → event autoarchive after its date.
 - Business-rule regression tests: quarterly submission limit, star-locking after approval, Blood Donation quarter exemption, rejection notification carrying the exact reason.
- Suites run in CI on every backend/frontend change; results tracked as pass/fail with a report artifact per run.

19.2 Performance/Load Testing — JMeter

- JMeter test plans targeting the highest-traffic endpoints:
 - GET /api/events (notice board) under concurrent volunteer load.
 - POST /api/submissions (photo upload) under concurrent submission load.
 - GET /api/admin/submissions (review queue) as volunteer counts grow.
- Target defined in Section 12: these endpoints should respond in under 2 seconds at a few-thousand-user scale. JMeter plans should be re-run before each major release and whenever the volunteer base grows significantly.